# KAPASA MAKASA UNIVERSITY

SCHOOL OF APPLIED SCIENCE AND OPEN LEARNING

LAB WORK

GROUP MEMBERS

| NAMES | STUDENT NUMBERS |
|---|---|
| MARY CHANDA | 20231336 |
| GABRIAL HICHINDU | 20231919 |
| NSHIMUNYA FAITH MOONO | 20231552 |
| NELSON KAPINGA | 20231839 |
| NAKAUNDI NATASHA | 20230173 |
| OWEN PHIRI | 20231625 |
| TAPELA ZIYELA | 20232005 |
| BEN ZULU | 20231886 |
| ELVIS SONKWE | 20231254 |
| JEFF SHICHILONGO | 20231327 |
| ROSARIA PHIRI | 20230145 |

PROGRAM: CYBER SECURITY

LECTURER: MR DERRICK DARRY

YEAR OF STUDY: 4th YEAR

DUE DATE: 19th AUGUST, 2026.

LAB 7: ANDROID SECURITY ASSESSMENT WITH DROZER.

TEST DEVICE: INFINIX SMART 10 HD – PHYSICAL ANDROID DEVICE

Executive Summary

This report presents the complete hands-on execution of Lab 7, “Android Security Assessment with the Drozer Framework.” The assessment used Drozer to examine the Sieve password-manager application and its Android inter-process communication (IPC) attack surface. The work covered environment preparation, installation and configuration, package enumeration, exported-component discovery, activity testing, content-provider testing, SQL-injection scanning, and broadcast-receiver enumeration.

The assessment identified two confirmed security weaknesses in the tested build. First, the exported PWList activity could be launched directly, allowing the application’s password-list screen to be reached without following the normal authentication flow. Second, the exported /Passwords content-provider path disclosed stored account information without authentication. A third, medium-severity issue was identified in the provider’s access-control design: the /Keys path was protected with dedicated permissions, while /Passwords lacked equivalent protection.

The assessment also recorded two important negative results. Drozer’s automated provider SQL-injection scan reported no vulnerabilities in the tested projection or selection clauses, and the application exposed no exported broadcast receivers on this build. These results are retained because they show that the assessment was evidence-driven rather than based on assuming that every historically documented Sieve weakness was present.

Key Findings at a Glance

| ID | Findings | Severity | Component |
|---|---|---|---|
| F1 | Authentication bypass through directly exported PWList activity | High | PWlist |
| F2 | Unauthenticated disclosure through /Passwords provider path | High | DBContentProvider |
| F3 | Inconsistent provider access control: /Keys protected, /Passwords exposed | Medium | DBContentProvider |
| F4 | No SQL injection identified by automated provider scan | Informational | DBContentProvider / FileBackupProvider |
| F5 | No exported broadcast receivers present on this build | informational | N/A |

# 1. Introduction and Objective

The purpose of this lab was to conduct a structured security assessment of an Android application using Drozer, an open-source framework for testing Android applications and their IPC interfaces. The assessment focused on components that may be reachable by other applications, including activities, services, content providers, and broadcast receivers.

The target was Sieve, an intentionally vulnerable password manager created for Drozer security testing. Unlike the emulator-based environment assumed by the original lab manual, this assessment was performed against a physical Android device. That difference introduced compatibility issues that had to be resolved during setup and are documented below.

## 1.1 Learning Objectives

- Install and configure Drozer for Android security testing.
- Identify the attack surface of an Android application.
- Exploit vulnerabilities in exported components.
- Assess Inter-Process Communication (IPC) security.
- Generate a structured security assessment report.

## 2. Assessment Environment

The environment below reflects the actual configuration used during the assessment. The physical-device setup is significant because the original manual assumes an emulator and older Android tooling.

| Component | Actual Configuration |
|---|---|
| Host OS | Windows 10/11 (build 10.0.26200) |
| Test device | INFINIX Infinix X6525D, physical Android device |
| Python | Python 3.14, installed via python.org installer |
| Drozer Console | v3.1.0, installed via pip |
| Drozer Agent | v3.1.0, installed on the Android device via ADB |
| Target application | Sieve v1.0 (com.mwr.example.sieve) |
| ADB | Android Debug Bridge 1.0.41 (platform-tools 37.0.1) |

## 3. Setup Procedure

The setup process moved from a Windows host with no working Drozer session to a connected Drozer console communicating with the Agent on the physical Android device.

## 3.1 Installing the Drozer Console

The Drozer console was installed on the Windows host using pip:

```
pip install drozer
```

The package manager confirmed that Drozer and its required dependencies, including pyOpenSSL, Twisted, service-identity, and cryptography, were satisfied.

```
Requirement already satisfied: pyopenssl>=22.0.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from drozer) (26.4.0
)
Requirement already satisfied: twisted>=18.9.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from drozer) (26.4.0)
Requirement already satisfied: service-identity in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from drozer) (26.1.0)
Requirement already satisfied: distro in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from drozer) (1.9.0)
Requirement already satisfied: pyyaml in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from drozer) (6.0.3)
Requirement already satisfied: cryptography<51,>=49.0.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from pyopenss
l>=22.0.0->drozer) (50.0.0)
Requirement already satisfied: cffi>=2.0.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from cryptography<51,>=49.
0.0->pyopenssl>=22.0.0->drozer) (2.1.1)
Requirement already satisfied: pycparser in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from cffi>=2.0.0->cryptograp
hy<51,>=49.0.0->pyopenssl>=22.0.0->drozer) (3.0)
Requirement already satisfied: attrs>=22.2.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from twisted>=18.9.0->dr
ozer) (26.1.0)
Requirement already satisfied: automat>=24.8.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from twisted>=18.9.0->
drozer) (25.4.16)
Requirement already satisfied: constantly>=15.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from twisted>=18.9.0-
>drozer) (23.10.4)
Requirement already satisfied: hyperlink>=17.1.1 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from twisted>=18.9.0
->drozer) (21.0.0)
Requirement already satisfied: incremental>=24.7.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from twisted>=18.9
.0->drozer) (24.11.0)
Requirement already satisfied: typing-extensions>=4.2.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from twisted>
=18.9.0->drozer) (4.16.0)
Requirement already satisfied: zope-interface>=5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from twisted>=18.9.0
->drozer) (8.5)
Requirement already satisfied: idna>=2.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from hyperlink>=17.1.1->twis
ted>=18.9.0->drozer) (3.18)
Requirement already satisfied: packaging>=17.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from incremental>=24.7
.0->twisted>=18.9.0->drozer) (26.2)
```

*Figure 1 — pip output confirming that Drozer and its dependencies were installed.*

## 3.2 Downloading the Target Files

The original download links referenced by the lab manual were no longer available. The working release files used for this assessment were obtained from the mwrlabs/drozer GitHub release mirror.

```
curl -L -o sieve.apk https://github.com/mwrlabs/drozer/releases/download/2.3.4/sieve.apk
```

```
C:\Users\mccha>curl -L -o sieve.apk https://github.com/mwrlabs/drozer/releases/download/2.3.4/sieve.apk
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0      0   0      0   0      0      0      0          00:01              0
  0      0   0      0   0      0      0      0          00:01              0
100 359.2k 100 359.2k   0      0 105.7k      0   00:03   00:03         285.4k

C:\Users\mccha>
```

*Figure 2 — Downloading the Sieve APK (sieve.apk).*

```
curl -L -o drozer-agent.apk https://github.com/mwrlabs/drozer/releases/download/2.4.4/drozer-agent-2.4.4.apk
```

```
C:\Users\mccha>curl -L -o drozer-agent.apk https://github.com/mwrlabs/drozer/releases/download/2.4.4/drozer-agent-2.4.4.apk
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0      0   0      0   0      0      0      0          00:01              0
100      9 100      9   0      0      5      0   00:01   00:01              0

C:\Users\mccha>
```

*Figure 3 — Downloading the Drozer Agent APK (drozer-agent-2.4.4.apk).*

### 3.3 Installing the Java Runtime

A Java runtime was required by the Drozer environment and was not initially present. Eclipse Temurin JDK 25.0.4 was installed with the options to set JAVA_HOME and add Java to the system PATH enabled.

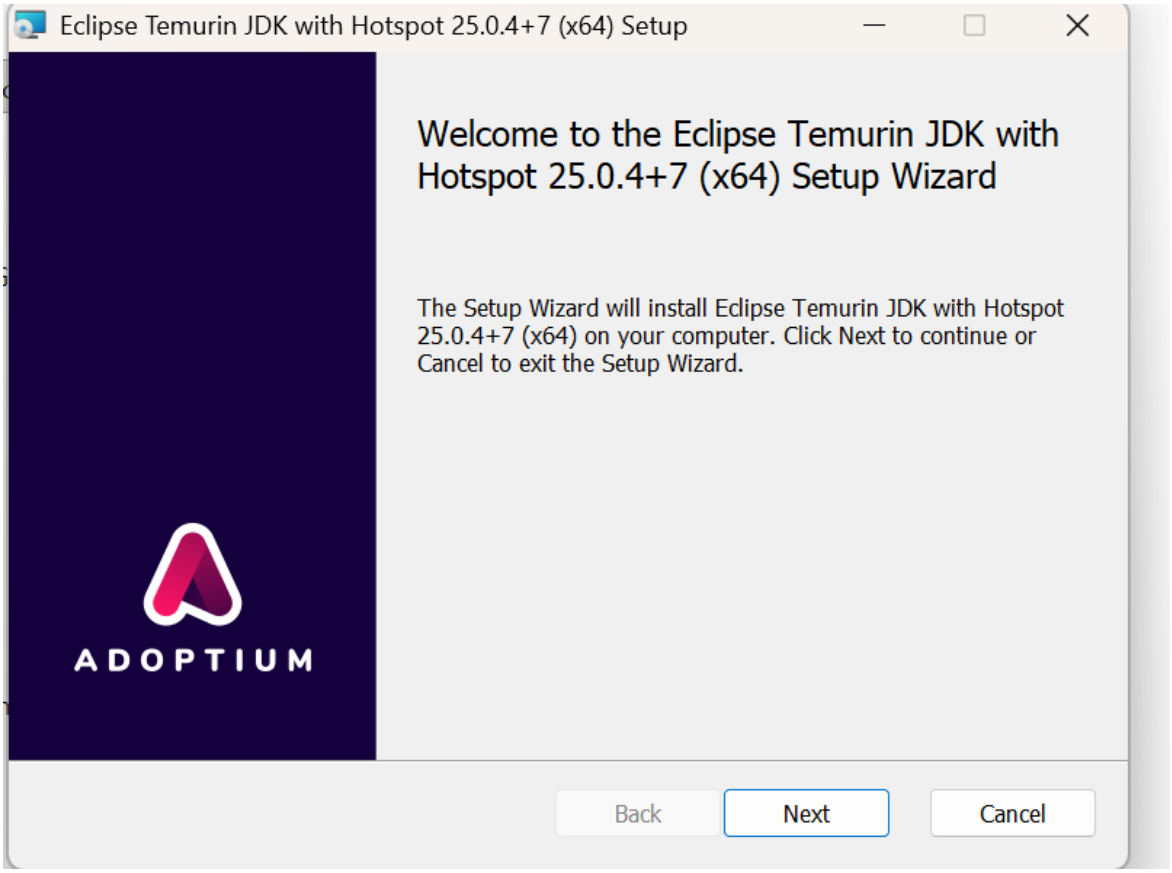



*Figure 4 — Eclipse Temurin JDK 25 installer.*

After installation and reopening the terminal, Java was verified with:

```
java -version
```

The reported version was OpenJDK 25.0.4, Temurin-25.0.4+7 (LTS).

### 3.4 Installing the Drozer Agent and Sieve

The first installation attempt failed because both Sieve and the older Drozer Agent target Android SDK 17. On the modern physical device, Android rejected these legacy-targeted packages with INSTALL_FAILED_DEPRECATED_SDK_VERSION.

The issue was resolved using the ADB flag intended to permit installation of legacy-targeted applications for testing:

```
adb install --bypass-low-target-sdk-block drozer-agent.apk
adb install --bypass-low-target-sdk-block sieve.apk
```




*Figure 5 — Successful installation of the Drozer Agent using the low-target-SDK bypass.*

The installed platform-tools version was 37.0.1, which supported the bypass option used above.

### 3.5 Configuring Port Forwarding

The Drozer Agent listens on TCP port 31415. ADB forwarding mapped the same device port to the host over USB:

```
adb forward tcp:31415 tcp:31415
```

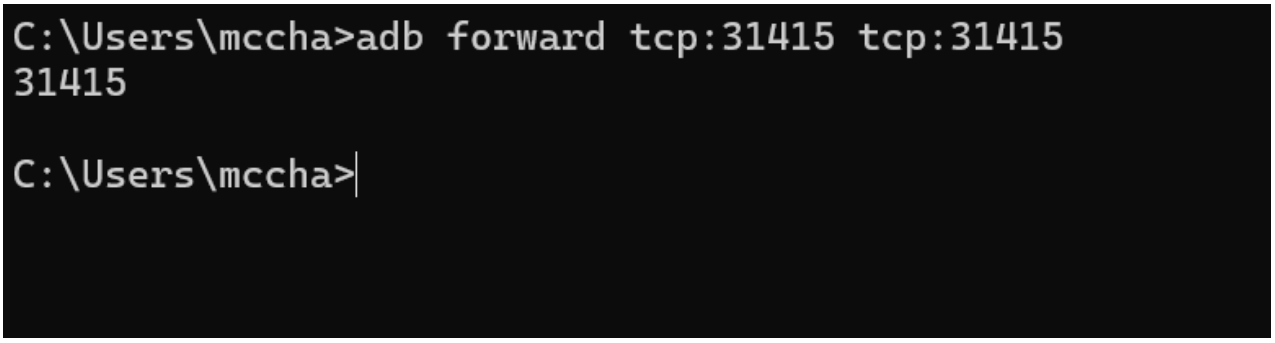



*Figure 6 — ADB port forwarding established for TCP port 31415.*

### 3.6 Connecting the Drozer Console

With the Drozer Agent open on the phone and its Embedded Server enabled, the Drozer console was started on Windows. Because the pip installation did not automatically place drozer.exe on the system PATH, the executable was launched using its full path.

```
"C:\Users\mccha\AppData\Local\Programs\Python\Python314\Scripts\drozer.exe" console connect
```

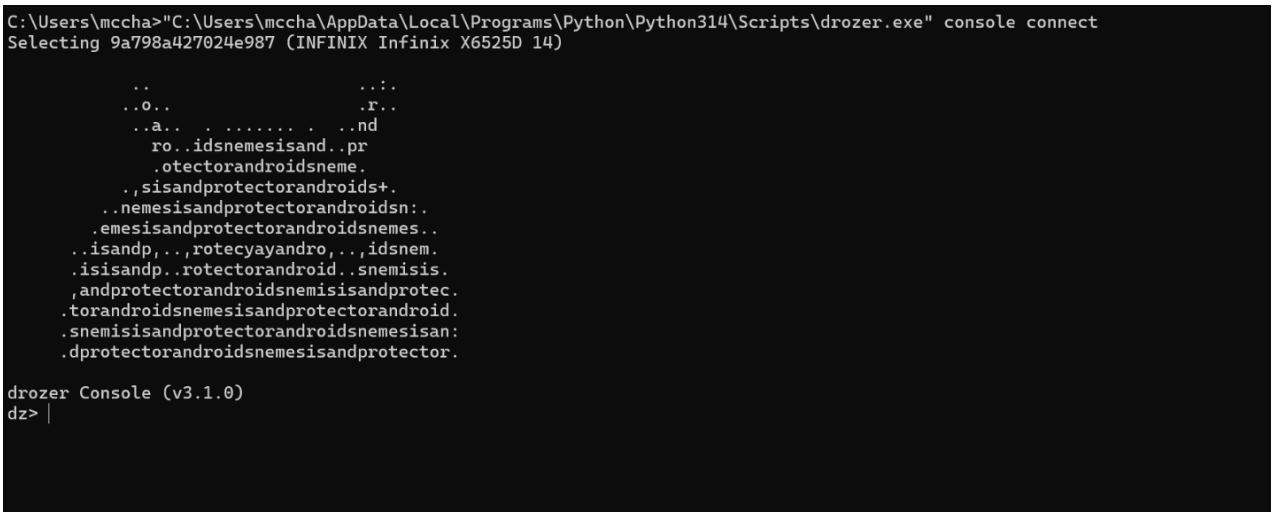



*Figure 7 — Drozer console successfully connecting to the physical INFINIX device and reaching the dz> prompt.*

Reaching the dz> prompt confirmed that the host console could communicate with the Drozer Agent. The assessment could therefore proceed to Parts A through E.

## 4. Part A Package Enumeration

The first assessment step was to identify the exact package name and inspect the application's declared permissions. This establishes the correct target identifier for all later Drozer commands and provides an initial view of the application's security model.

```
dz> run app.package.list -f sieve
dz> run app.package.info -a com.mwr.example.sieve
```

```
drozer Console (v3.1.0)
dz> run app.package.list -f sieve
Attempting to run shell module
com.mwr.example.sieve (Sieve)
dz> run app.package.info -a com.mwr.example.sieve
Attempting to run shell module
Package: com.mwr.example.sieve
  Application Label: Sieve
  Process Name: com.mwr.example.sieve
  Version: 1.0
  Data Directory: /data/user/0/com.mwr.example.sieve
  APK Path: /data/app/~~AsRIvWITIflpbXJtMqnREg==/com.mwr.example.sieve-AdFKYP-8E7vRN2924oA06A==/base.apk
  UID: 10230
  GID: [3003]
  Shared Libraries: [/system/framework/android.test.base.jar, /system/framework/org.apache.http.legacy.jar]
  Shared User ID: null
  Uses Permissions:
  - android.permission.READ_EXTERNAL_STORAGE
  - android.permission.WRITE_EXTERNAL_STORAGE
  - android.permission.INTERNET
  - android.permission.POST_NOTIFICATIONS
  - android.permission.ACCESS_MEDIA_LOCATION
  - android.permission.READ_MEDIA_AUDIO
  - android.permission.READ_MEDIA_VIDEO
  - android.permission.READ_MEDIA_IMAGES
  - android.permission.READ_MEDIA_VISUAL_USER_SELECTED
  Defines Permissions:
  - com.mwr.example.sieve.READ_KEYS
  - com.mwr.example.sieve.WRITE_KEYS
```

*Figure 8 — Package enumeration and application information for com.mwr.example.sieve.*

The output identified the target as com.mwr.example.sieve, version 1.0, running under UID 10230. The application requested storage, Internet, and media-related permissions. More importantly, the application declared two custom permissions: com.mwr.example.sieve.READ_KEYS and com.mwr.example.sieve.WRITE_KEYS. These permissions became relevant when the DBContentProvider was examined later.

## 5. Part B — Attack Surface Identification

The attack-surface command identifies exported Android components that may be reachable by other applications on the same device without requiring a special privilege.

```
dz> run app.package.attacksurface com.mwr.example.sieve
```

```
dz> run app.package.attacksurface com.mwr.example.sieve
Attempting to run shell module
Attack Surface:
  3 activities exported
  0 broadcast receivers exported
  2 content providers exported
  2 services exported
    is debuggable
dz>
```

*Figure 9 — Drozer attack-surface enumeration for com.mwr.example.sieve.*

The tested build reported three exported activities, zero exported broadcast receivers, two exported content providers, two exported services, and a debuggable application.

Version note: These counts differ from the older lab manual's reference output (2 activities, 1 broadcast receiver, 1 content provider, and 1 service). The report attributes the difference to the newer Sieve build and Drozer Agent version used for this assessment. The absence of exported broadcast receivers is therefore treated as a tested null result rather than an omitted task.

## 6. Part C — Activity Exploitation

The three exported activities were reviewed to determine whether they could be launched directly, without first passing through the application's intended login flow.

```
dz> run app.activity.info -a com.mwr.example.sieve
```

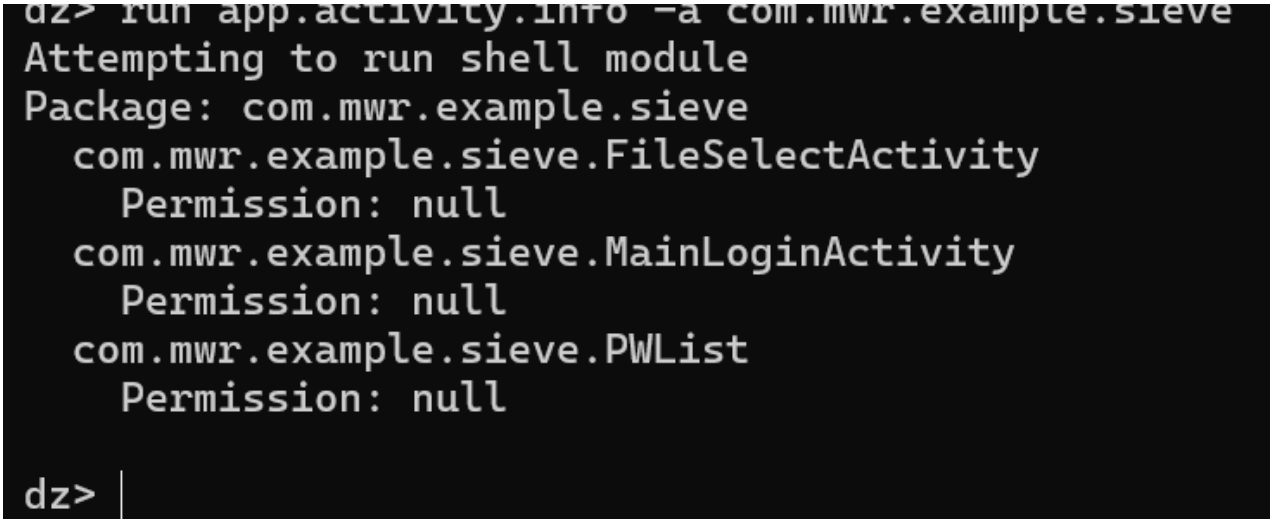



*Figure 10 — Exported activities and their permission requirements.*

FileSelectActivity, MainLoginActivity, and PWList were all shown as exported with no required permission. PWList is particularly sensitive because it is the screen that displays the user's stored passwords.

```
dz> run app.activity.start --component com.mwr.example.sieve com.mwr.example.sieve.MainLoginActivity
```

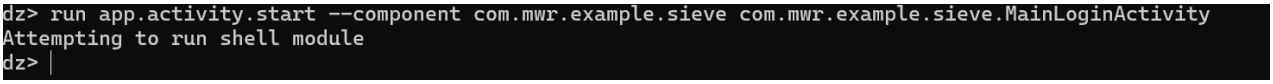



*Figure 11 — MainLoginActivity launched directly through Drozer.*

A separate test targeted PWList directly rather than MainLoginActivity:

```
dz> run app.activity.start --component com.mwr.example.sieve com.mwr.example.sieve.PWList
```

According to the documented test result, launching PWList directly opened the password-list screen without requesting the PIN or master password. This demonstrates that the activity relied on the user having authenticated through MainLoginActivity rather than enforcing authentication itself.

> FINDING F1 — HIGH: Authentication bypass via exported activity. PWList was exported and did not require a permission. Because Android permits another application to start an exported activity directly, the application's intended navigation order could be bypassed.

Recommended remediation: set android:exported="false" on PWList unless external access is genuinely required. If the activity must remain externally reachable, it should verify authentication/session state in its own lifecycle, such as onCreate(), rather than depending solely on the launcher activity.

7. Part D Content Provider Assessment

The assessment next examined the two exported content providers identified in Part B: DBContentProvider and FileBackupProvider. The purpose was to determine which URI paths responded and whether sensitive information could be accessed without authentication.

```
dz> run scanner.provider.finduris -a com.mwr.example.sieve
```

```
drozer Console (v3.1.0)
dz> run scanner.provider.finduris -a com.mwr.example.sieve
Attempting to run shell module
Scanning com.mwr.example.sieve...
No respone from content URI:      content://com.mwr.example.sieve.DBContentProvider/
Got a response from content Uri:  content://com.mwr.example.sieve.DBContentProvider/Keys/
Got a response from content Uri:  content://com.mwr.example.sieve.DBContentProvider/Passwords
No respone from content URI:      content://com.mwr.example.sieve.FileBackupProvider
No respone from content URI:      content://com.mwr.example.sieve.FileBackupProvider/
No respone from content URI:      content://com.mwr.example.sieve.DBContentProvider
No respone from content URI:      content://com.mwr.example.sieve.DBContentProvider/Keys
Got a response from content Uri:  content://com.mwr.example.sieve.DBContentProvider/Passwords/

For sure accessible content URIs:
  content://com.mwr.example.sieve.DBContentProvider/Keys/
  content://com.mwr.example.sieve.DBContentProvider/Passwords
  content://com.mwr.example.sieve.DBContentProvider/Passwords/
```

*Figure 12 — Drozer URI scan identifying responsive content-provider paths.*

Three URIs responded during the scan: DBContentProvider/Keys/, DBContentProvider/Passwords, and DBContentProvider/Passwords/. No tested path under FileBackupProvider responded.

7.1 Querying the Passwords Provider

To produce a realistic test record, one entry was created through Sieve's normal interface after authentication. The documented test entry used the service name "Test Bank," username "testuser," password "Testpass123," and email "test@example.com." The provider was then queried directly through Drozer, bypassing the application's normal login interface.

```
dz> run app.provider.query content://com.mwr.example.sieve.DBContentProvider/Passwords/
```

```
dz> run app.provider.query content://com.mwr.example.sieve.DBContentProvider/Passwords/
Attempting to run shell module
| _id | service   | username | password                                                | email            |
| 1   | Test Bank | testuser | b'Y9/dgL6obBz33z0QuTH1ZV+r9W5rByy8unF8' (Base64-encoded) | test@example.com |

dz>
```

*Figure 13 — Test password record retrieved through the exported /Passwords provider path.*

The /Passwords path returned service name, username, email, and the password field without an authentication check. The password field was returned as Base64-encoded data rather than readable plaintext.

The report's documented analysis states that decoding the Base64 value produced non-readable binary data, indicating that the stored password was likely encrypted before being Base64-

encoded. Therefore, the provider did not trivially reveal the original plaintext password from this query alone; however, it still exposed account metadata and the encrypted password blob to an unauthenticated caller.

> FINDING F2 — HIGH: Unauthenticated data disclosure via exported content provider. The /Passwords path exposed sensitive account information without access control. The provider should be protected with an appropriate read/write permission pair or made non-exported if external access is not required.

### 7.2 Automated SQL Injection Scan

Drozer provides an automated provider scanner that tests discovered URIs for SQL injection through both projection and selection clauses.

```
dz> run scanner.provider.injection -a com.mwr.example.sieve
```

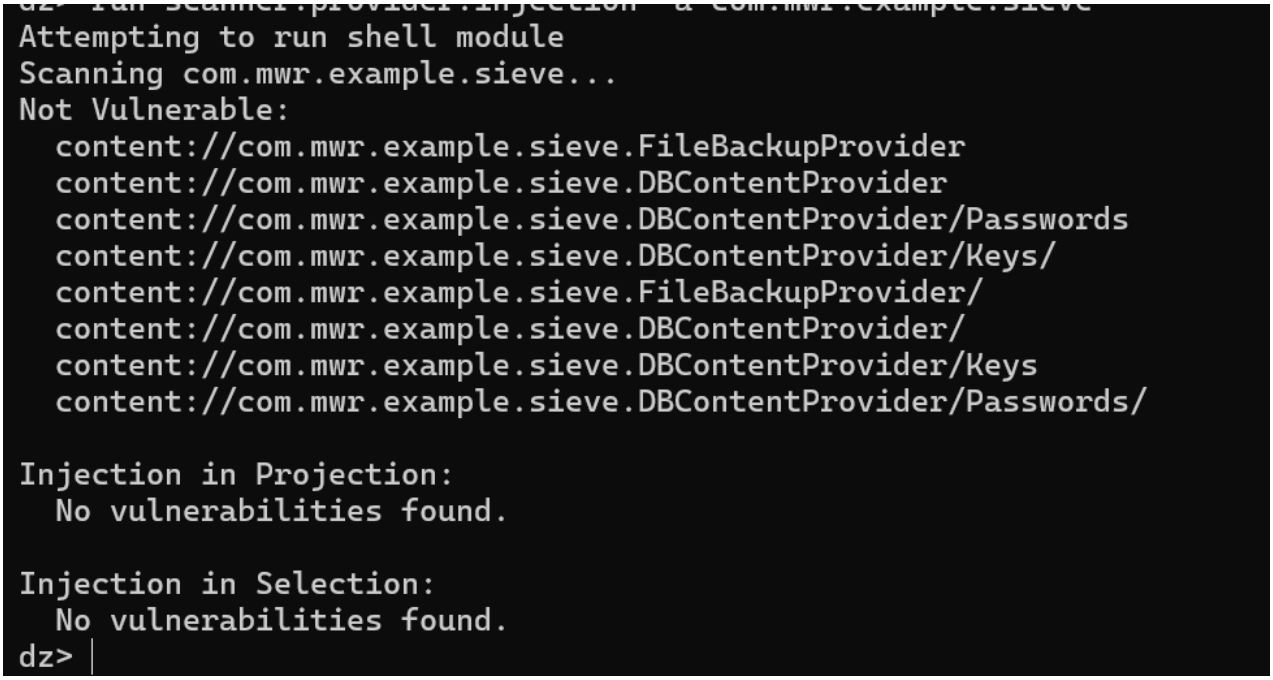



*Figure 14 — Automated SQL-injection scan results.*

RESULT — Informational: No SQL injection was reported in the tested projection or selection clauses for the discovered provider URIs. This negative result is retained as part of the assessment evidence and does not invalidate the two confirmed weaknesses above.

### 7.3 Comparing Protection: The Keys Provider

The /Keys path was then examined because it stores the application's master password and PIN used for its internal authentication.

```
dz> run app.provider.query content://com.mwr.example.sieve.DBContentProvider/Keys/
```

The documented primary test result states that this query failed with “Could not get a ContentProviderClient,” which was interpreted as a permission denial rather than a connection failure. The same Drozer session remained functional for other package and provider queries.

```
dz> run app.provider.info -a com.mwr.example.sieve
```

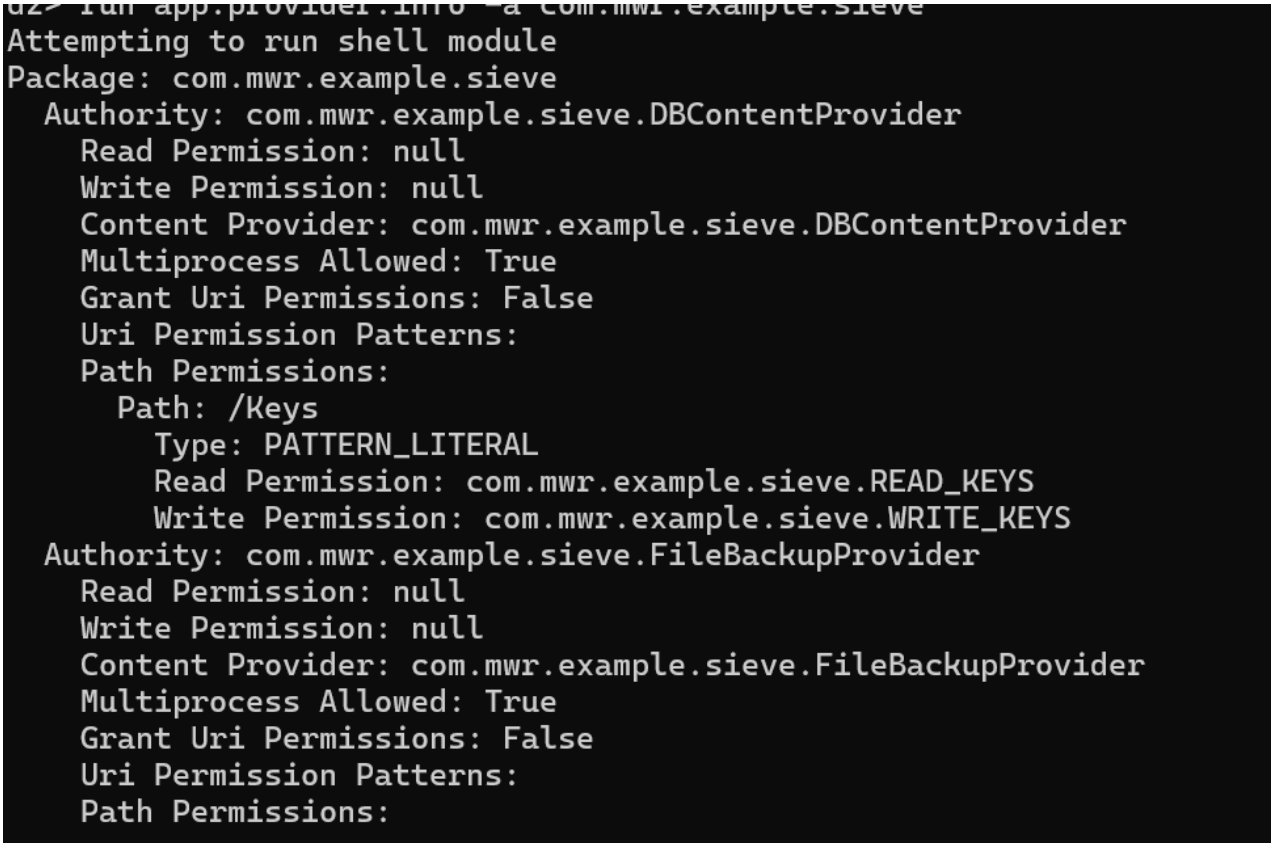



*Figure 15 — Provider-level and path-level permissions reported for DBContentProvider.*

The provider information showed that /Keys was protected by the dedicated permissions com.mwr.example.sieve.READ_KEYS and com.mwr.example.sieve.WRITE_KEYS. The /Passwords path did not have equivalent path-level protection.

> FINDING F3 — MEDIUM: Inconsistent access control across provider paths. The application demonstrated that sensitive provider data could be protected with path-level permissions, but that protection was not applied consistently to /Passwords.

## 8. Part E — Broadcast Receiver Exploitation

The attack-surface scan reported zero exported broadcast receivers. To verify the result directly, the broadcast receiver enumeration command from the lab manual was executed.

```
dz> run app.broadcast.info -a com.mwr.example.sieve
```

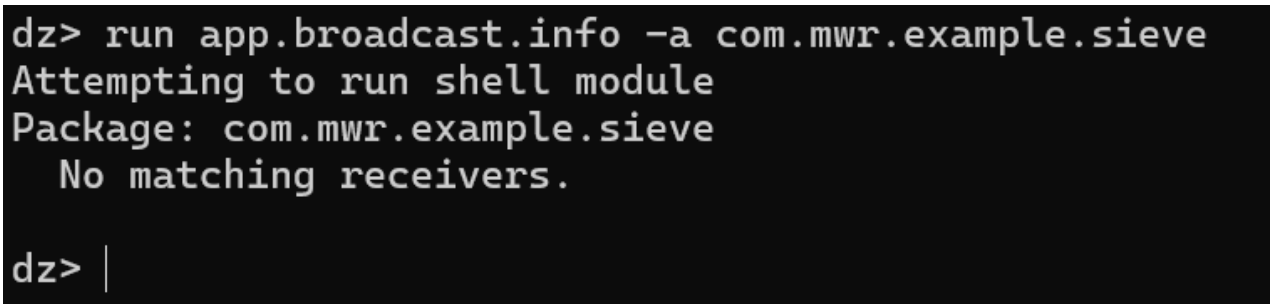



*Figure 16 — Broadcast receiver enumeration confirming that no matching exported receivers were present.*

RESULT — Informational: The command returned "No matching receivers." No broadcast receiver was therefore available for the broadcast-based exploitation procedure on this build. This is recorded as a legitimate null result rather than as a skipped lab requirement.

## 9. Summary of Findings

The assessment identified a coherent set of weaknesses centered on Android component exposure and IPC access control. The most significant issue was the ability to launch PWList directly, bypassing the application's intended authentication sequence. The second major issue was unauthenticated access to the /Passwords content-provider path. Together, these findings show that sensitive application functionality and data were exposed through exported components.

| # | Finding | Severity | Assessment Result |
|---|---|---|---|
| 1 | Authentication bypass via exported PWList activity | High | Confirmed |
| 2 | Unauthenticated disclosure via /Passwords | High | Confirmed |
| 3 | Inconsistent provider access control | Medium | Confirmed |
| 4 | SQL injection in provider projection/selection | Informational | Not found |
| 5 | Exported broadcast receivers | Informational | None present |

Overall assessment: the tested build is not universally vulnerable. The confirmed weaknesses are specific to exported component exposure and inconsistent access-control enforcement. The absence of SQL injection and exported broadcast receivers is therefore an important part of the final security picture.

## 10. Conclusion and Remediation Recommendations

The assessment met the five learning objectives defined for the lab. Drozer was installed and connected end-to-end; the Sieve attack surface was enumerated; exported-component

vulnerabilities were tested; IPC permissions were examined; and the results were consolidated into a security assessment report.

10.1 Recommended Remediations

| Priority | Control | Recommendation |
|---|---|---|
| | Restrict unnecessary activity exposure. | Set android:exported="false" on PWList, FileSelectActivity, and MainLoginActivity unless external access is a genuine product requirement. |
| | Enforce authentication at the sensitive component. | If an activity must remain exported, verify authentication/session state inside that activity's own lifecycle, such as onCreate(). |
| | Protect the /Passwords provider path. | Apply a dedicated read/write permission pair to /Passwords, following the protection model already used for /Keys. |
| | Disable provider export where possible. | Consider setting android:exported="false" on DBContentProvider if other applications do not legitimately need access to it. |
| | Protect sensitive stored data. | Continue using strong encryption for sensitive password fields and ensure the corresponding decryption key cannot itself be exposed through an unauthenticated channel. |

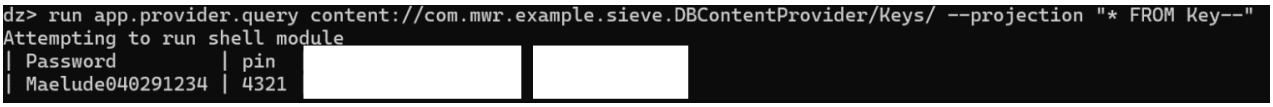
10.2 Scope Statement

The assessment was limited to the Sieve training application (com.mwr.example.sieve) running on the identified physical Android device. No production, third-party, or unauthorized systems were targeted. Results describe the observed behavior of the tested Sieve build and should not automatically be generalized to other versions.Appendix

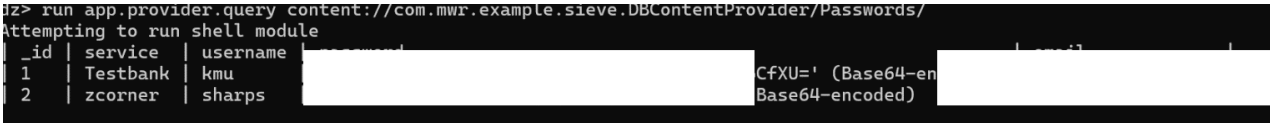
A Supplementary Evidence Reconciliation

Two additional screenshots were present at the end of the source report without figure captions or supporting narrative. They are retained here as supplementary evidence rather than incorporated into the primary findings because their displayed command/results do not fully align with the documented narrative in Section 7.3.

> Important: The original screenshots contain credential-like values. The report copy below redacts those values for safe presentation. Before final academic submission, the original evidence should be reviewed to determine whether these captures came from a separate test, a different provider-query method, or an earlier test state.




*Appendix Figure A1 Supplementary /Keys provider-query screenshot; sensitive returned values redacted.*




*Appendix Figure A2 Supplementary /Passwords provider-query screenshot; sensitive data values redacted.*

Reconciliation note: the primary report records the /Keys query as denied and the /Passwords query as unauthenticated data disclosure. Because the supplementary screenshots appear to show additional provider-query output, they should be reconciled against the original lab session before making any new claim about key or PIN disclosure.

Appendix B — Evidence and Reproducibility Checklist

- Drozer Console installed and operational on the Windows host.
- Drozer Agent installed and connected on the physical Android device.
- ADB port 31415 forwarded successfully.
- Target package confirmed as com.mwr.example.sieve.
- Attack surface enumerated with Drozer.
- Exported activities enumerated and direct-launch testing performed.
- Content-provider URIs enumerated.
- Unauthenticated /Passwords provider query tested.
- Automated provider SQL-injection scan completed.
- /Keys provider protection inspected.
- Broadcast receiver enumeration completed.
- Findings consolidated with severity and remediation recommendations.